

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 04 -

AMBIENTES REPRODUTÍVEIS COM DOCKER

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	20
O QUE VOCÊ VIU NESTA AULA?	21
REFERÊNCIAS.....	22

EMSE

O QUE VEM POR AÍ?

É comum que códigos de ciência de dados funcionem em um ambiente, mas falhem em outro por diferenças em dependências ou sistemas. O Docker resolve esse problema ao empacotar aplicações e todas as suas dependências em contêineres isolados e padronizados, garantindo o mesmo funcionamento em diferentes máquinas.

Isso facilita a reprodutibilidade, a colaboração entre equipes e a portabilidade entre ambientes de desenvolvimento e produção. Nesta aula, vamos entender como criar imagens Docker, executar contêineres, aplicar boas práticas e analisar casos reais no contexto da ciência de dados.

HANDS ON

Nesta etapa prática da aula, você irá acompanhar as videoaulas nas quais mostramos, de forma aplicada, como criar ambientes reprodutíveis para projetos de ciência de dados utilizando o Docker. O objetivo deste hands on é proporcionar uma experiência real de implementação, permitindo que você veja, na prática, o processo de empacotar um projeto Python com todas as suas dependências, facilitando o compartilhamento e a execução em diferentes computadores sem complicações técnicas.

As gravações detalham cada etapa, trazendo exemplos reais, instruções passo a passo e dicas para evitar problemas comuns no cotidiano de quem trabalha com dados.

Para ilustrar, vamos criar um ambiente Docker para um projeto simples de análise de dados em Python, que utiliza bibliotecas como pandas e numpy. O ponto de partida é a escrita de um Dockerfile, em que definimos a imagem base (FROM python:3.9-slim), o diretório de trabalho (WORKDIR /app), a cópia dos arquivos necessários (COPY requirements.txt ./ e COPY . .), a instalação das dependências (RUN pip install -r requirements.txt) e a configuração do comando padrão (CMD ["python", "analise.py"]).

Com o Dockerfile pronto, mostramos como construir a imagem com docker build -t meu-projeto:1.0 . e, em seguida, como executar o contêiner de forma a garantir a persistência dos dados utilizando o comando docker run -it --name analise-container -v \$(pwd)/dados:/app/dados meu-projeto:1.0.

Durante as videoaulas serão destacados pontos importantes como boas práticas para manter imagens leves e seguras, a importância de fixar versões de dependências e de não inserir informações sensíveis no Dockerfile. Links de referência e exemplos estão disponíveis para consulta, além dos comandos demonstrados, facilitando o acompanhamento e a aplicação do conteúdo apresentado.

Ao final, você terá compreendido o fluxo completo, desde a preparação do ambiente até a execução do contêiner, pronto para adaptar a solução ao seu próprio projeto de ciência de dados.

SAIBA MAIS

Até aqui vimos na prática como contêineres Docker podem encapsular um projeto de ciência de dados, mas para um entendimento completo precisamos mergulhar em conceitos teóricos e técnicos por trás de Docker e contêineres. Vamos explorar em detalhes o que difere um contêiner de uma máquina virtual tradicional; como o Docker isola e controla recursos (CPU, memória, GPU); como contêineres podem ser escalados e distribuídos em clusters; e quais fundamentos de engenharia de software e sistemas operacionais sustentam essa tecnologia. Tudo isso contextualizando sempre para casos de ciência de dados – em que performance, gerenciamento de memória e reprodutibilidade são cruciais.

Contêineres vs. Máquinas Virtuais: virtualização leve

Uma dúvida comum ao iniciar é: “Contêiner não é só uma VM?”. Embora ambos providenciem isolamento, a forma como o fazem é bem diferente. Máquinas virtuais (VMs) tradicionalmente usam hipervisores para virtualizar o hardware inteiro: cada VM roda um sistema operacional completo (kernel + user space) em cima de um hardware virtualizado, o que consome bastante recurso.

Já um contêiner Docker realiza uma virtualização a nível de sistema operacional (OS-level virtualization), compartilhando o mesmo kernel do host, porém isolando processos, sistema de arquivos e rede através de primitivas do próprio OS (no caso do Docker, primitivas do kernel Linux). Na prática, um contêiner pode ser visto como um processo isolado rodando no host, com sua própria visão do sistema (como se tivesse seu próprio root filesystem, seus próprios processos PID 1, sua interface de rede independente etc.).

Isso faz com que contêineres sejam muito mais leves – não carregam um sistema operacional inteiro dentro de si, apenas usam partes necessárias, aproveitando o kernel já em execução. Uma VM leva minutos para iniciar; um contêiner sobe em segundos (ou menos) devido a essa leveza. Em termos de overhead, contêineres gastam bem menos CPU e memória para o isolamento.

Como bem resumiu Vinícius Victorino (2022), uma VM tradicional isola executando outro sistema operacional completo, consumindo muito mais recursos, enquanto contêineres isolam aplicações como processos do mesmo SO do host, de forma muito mais enxuta.

Tecnicamente, o Docker depende de duas funcionalidades-chave do Linux: namespaces e cgroups (grupos de controle). Namespaces implementam o isolamento: cada contêiner ganha um conjunto de namespaces próprios – por exemplo, PID namespace isolando sua árvore de processos, NET namespace isolando seus dispositivos de rede virtuais, MNT namespace isolando seu sistema de arquivos montado etc.

Isso garante que um processo dentro do contêiner tenha PID 1 no contexto dele (como se fosse init do “mini-sistema”), mas na verdade é apenas um processo normal do ponto de vista do kernel global. Já os cgroups cuidam do gerenciamento de recursos, permitindo impor limites e quotas de CPU, memória, I/O etc. para grupos de processos. O Docker cria cgroups para cada contêiner conforme necessário – por exemplo, se definirmos que um contêiner pode usar no máximo 2 GB de RAM, o kernel aplicará esse cgroup limitando a alocação de memória daquele grupo de processos.

Assim, mesmo que dentro do contêiner haja um pânico de consumo de memória, ele não derrubará o host; o kernel pode inclusive matar processos do contêiner (gera um OOM kill) se excederem o limite, protegendo o sistema. Por padrão, se não estipularmos nada, o contêiner pode usar recursos conforme disponível no host (ou seja, sem limites específicos além do compartilhamento padrão do SO). Mas o Docker nos dá a opção de setar flags no docker run para controle fino – `-m/--memory` para limite de RAM, `--cpus` ou `-c (shares)` para controlar CPU etc., com suporte do kernel Linux através dos cgroups para fazer cumprir esses limites.

Em resumo, contêineres são implementados como ambientes isolados no espaço do usuário, usando o mesmo kernel do host. Isso explica por que contêineres Linux rodam nativamente só em hosts Linux (ou em VMs Linux se o host for Windows/Mac, o que ferramentas como Docker Desktop fazem nos bastidores). No entanto, existem contêineres Windows análogos (embora menos populares em DS) que usam kernel Windows e funcionam apenas em hosts Windows.

A padronização vem da especificação OCI (Open Container Initiative), que define formatos e runtime de contêiner, permitindo inclusive alternativas como o Podman (que roda contêineres rootless, sem daemon). Mas conceitualmente, todos aproveitam essa ideia de isolamento leve provido pelo OS – “máquinas virtuais de processo” em vez de virtualização completa de hardware.

Gerenciamento de recursos: CPU, memória e GPU nos contêineres

Para aplicações de ciência de dados, desempenho é importante – treinar um modelo pesado envolve muita CPU/GPU e memória. Como os contêineres lidam com isso? Já vimos que, por padrão, um contêiner pode usar todo recurso disponível, competindo de forma “justa” com outros processos do host. O Docker permite limitar e reservar recursos via cgroups: por exemplo, podemos rodar `docker run --memory=4g --cpus=2` para restringir a 4 GB de RAM e ~2 CPUs o contêiner. Esses limites duros garantem que, mesmo que o código dentro do contêiner tente usar mais (threads extras, mallochs enormes), o kernel irá contê-lo – evitando travar o host por falta de memória, por exemplo.

Em cenários de multiusuário ou vários contêineres simultâneos (como vários experimentos rodando em paralelo em um servidor), isso é essencial para isolar consumo. Também há como definir soft limits e reservas – por exemplo, `--memory-reservation` para dar preferência e CPU shares para peso relativo de CPU – quando vários contêineres competem, o scheduler pode dividir o tempo de acordo com esses pesos.

Um recurso especial é a GPU. Contêineres inicialmente não tinham suporte direto a GPU (porque a GPU não era visível no namespace de dispositivos), mas hoje é possível habilitar. Para usar GPUs NVIDIA, por exemplo, instala-se o NVIDIA Container Toolkit no host e executa-se o contêiner com `--gpus` (ex.: `docker run --gpus all ... nvidia/cuda:12.2-python3.10 ...`). Isso permite que o contêiner acesse as placas de vídeo do host para treinar modelos de Deep Learning, por exemplo. Internamente, o Docker monta os drivers necessários no contêiner e autoriza acesso aos dispositivos `/dev/nvidia*`.

Assim, conseguimos isolamento de software sem perder aceleração por hardware. Em cenários de clusters, também há orquestradores (como Kubernetes

com device plugins) que gerenciam a alocação de GPUs para contêineres. Vale ressaltar que o contêiner em si não virtualiza uma GPU – ele usa a GPU real do host – portanto, o desempenho dentro ou fora do contêiner é praticamente idêntico, já que o overhead de containerização é mínimo (perto de 2-5%). Essa leve perda vem principalmente do nível extra de isolamento, mas nada comparado a overheads de VM. Para CPU, I/O e memória, benchmarks mostram que o impacto de rodar no Docker versus nativamente é desprezível, o que torna contêineres muito atraentes: podemos encapsular sem abrir mão de desempenho.

Do ponto de vista de sistemas operacionais, o Docker se apoia em mecanismos de escalonamento e alocação do kernel. Contêineres repartem CPU de acordo com o scheduler Linux normal (CFS), a não ser que usemos afinidades ou limites. Já a memória, como mencionado, pode ser restringida. Se um contêiner consome memória descontroladamente e o host esgota RAM, o kernel Linux pode acionar o OOM Killer – frequentemente ele irá matar o processo do contêiner problemático antes de matar o próprio Docker ou outros serviços do sistema. Isso é bom: mantém a estabilidade do host. Podemos sintonizar esse comportamento (há flags avançadas como `--oom-kill-disable`, não recomendadas sem `-m`, pois desabilitar o OOM killer sem limitar memória pode levar o host ao colapso).

Em resumo, o Docker oferece controle refinado de recursos, o que é útil em cenários de ciência de dados compartilhados (ex.: vários cientistas rodando experimentos no mesmo servidor – cada um no seu container limitado para não atropelar o outro). Também é útil para simular ambientes de produção com menos recursos: por exemplo, testar se seu pipeline ML degradaria em uma máquina com apenas 1 CPU, limitando seu container a 1 CPU para ver o efeito.

Vale mencionar que contêineres herdam o kernel do host. Portanto, certos comportamentos são idênticos ao host – por exemplo, um contêiner Linux terá a mesma gestão de memória virtual do host (incluindo estratégias de paginação). Técnicas de alto desempenho, como uso de huge pages para memória compartilhada em treino de modelos, precisam ser suportadas pelo kernel host para refletir no contêiner. No entanto, o isolamento garante que dentro do contêiner a aplicação “enxerga” apenas os recursos alocados a ela. Ferramentas de monitoramento dentro do contêiner (como `free -m` ou `nproc`) reportam valores

correspondentes aos limites, não ao total do host, prevenindo confusão para a aplicação.

Em termos de implementação de baixo nível, além de cgroups e namespaces, o Docker usa Union File Systems para as imagens – cada camada do Dockerfile é uma sobreposição de sistema de arquivos somente-leitura, com o contêiner ativo possuindo uma camada de leitura-escrita no topo. Isso possibilita o reuso eficiente de camadas entre contêineres e rollback rápido, mas significa que a escrita intensiva na camada superior pode ter pequena penalização. Para dados pesados (como escrita de checkpoints de modelo), é recomendável usar volumes montados (que escrevem diretamente no host ou em disco externo) para performance nativa.

Escalabilidade e distribuição de contêineres

Um único contêiner rodando em uma máquina já resolve o “funciona em qualquer lugar”; mas e se quisermos lidar com grandes cargas de trabalho ou alta disponibilidade? Aí entra a possibilidade de escalar horizontalmente nossos ambientes Docker. Na prática, escalar contêineres significa rodar múltiplas instâncias de um mesmo contêiner (réplicas) distribuídas em um ou vários hosts e coordená-las. Diferentemente de processos tradicionais, contêineres têm a vantagem de serem facilmente replicáveis: como cada instância carrega todas as dependências, você pode levantar 10 contêineres idênticos da sua aplicação de ML sem temer diferenças entre eles.

Isso é muito usado em produção para servir modelos em larga escala – por exemplo, se um modelo de recomendação precisa servir milhões de requisições, você coloca várias réplicas do serviço em contêineres atrás de um balanceador de carga. Ferramentas de orquestração como Kubernetes automatizam esse processo de escalonamento.

O Kubernetes (K8s) é hoje a plataforma mais adotada para gerenciar contêineres em cluster. Conceitos-chave: você define um deployment para sua aplicação contêinerizada, especificando quantas réplicas quer e o K8s se encarrega de agendar esses contêineres (pods) nos nós disponíveis, monitorando e mantendo o estado desejado. Ele usa um escalonador (scheduler) interno que decide em qual máquina cada contêiner roda, considerando recursos disponíveis, afinidades, etc. –

é um problema de alocação de recursos que o K8s resolve com heurísticas (similar ao bin packing).

Para dimensionamento dinâmico, o Horizontal Pod Autoscaler (HPA) do K8s pode aumentar ou reduzir o número de contêineres de acordo com métricas de uso (CPU, memória ou métricas customizadas). Por exemplo, se o uso de CPU médio dos pods passar de 80%, o HPA pode subir mais instâncias; se cair abaixo de um limiar, remover algumas, tentando acompanhar a demanda. Já o Vertical Pod Autoscaler (VPA) ajusta recursos de cada contêiner (dando mais CPU/memória para um pod conforme necessidade) – mas para cargas de trabalho de ciência de dados, escalar horizontalmente costuma ser mais eficiente quando possível (treinamentos distribuídos etc.), salvo casos de componentes não paralelizáveis que precisam de mais memória. Em cenários de big data, contêineres são usados também para orquestrar frameworks como Spark em clusters – cada executor Spark pode rodar dentro de um contêiner, facilitando isolamento e reutilização de recursos no cluster.

Um conceito importante é o de balanceamento de carga. Quando temos múltiplos contêineres replicando um serviço (por exemplo, vários servidores de um modelo ML), precisamos distribuir as requisições entre eles. Ferramentas de orquestração costumam fornecer service discovery e load balancing. No Docker standalone, existe o Docker Swarm que tem recursos nativos de distribuir carga entre contêineres em cluster. No Kubernetes, um Service abstrai um conjunto de pods e pode rodar um proxy/balanceador (como kube-proxy) para rotear pedidos para os pods disponíveis. Os algoritmos de roteamento geralmente são de rodízio simples (round-robin) ou baseados em acaso, às vezes considerando o peso. O essencial é que contêineres tornam a escala horizontal trivial: se um pod cair, outro pode substituí-lo rapidamente (K8s cuida de reiniciar) e se precisarmos dobrar throughput, simplesmente dobramos o número de contêineres.

Para o processamento distribuído de dados, contêineres podem encapsular componentes de frameworks como Hadoop, Spark, Dask. Um exemplo: você pode ter um cluster Kubernetes rodando contêineres Spark – o driver e executors são contêineres separados. Empresas como Uber e Netflix implementaram suas plataformas de ML com containers; o Uber, por exemplo, no sistema Michelangelo (embora detalhes internos sejam proprietários, sabe-se que usam bastante Docker para isolamento em jobs agendados).

Nesses cenários, surge a necessidade de orquestrar centenas ou milhares de contêineres, onde surgem desafios de granularidade (quão grande cada contêiner deve ser em termos de tarefas?), eficiência de escalonamento e rede. Protocolos como MPI para treinamento distribuído também podem rodar em contêineres – cada nó MPI contido, mas comunicando via rede do cluster.

Do ponto de vista algorítmico, escalar horizontalmente tem limites teóricos dados pela Lei de Amdahl: se uma porção P do seu workload é paralelizável, em teoria escalar para N instâncias reduz o tempo nessa parte em até N vezes, mas sempre haverá a parcela sequencial $(1-P)$ que permanece como gargalo. Por isso, adicionar contêineres indefinidamente dá retornos decrescentes se o problema não paraleliza bem. Cientistas de dados devem estar cientes disso ao arquitetar soluções: às vezes melhorar o algoritmo (aumentar P) vale mais do que só jogar mais contêineres.

Ainda assim, contêineres facilitam alcançar o speedup ideal porque removem variáveis de ambiente – cada instância paralela é idêntica. Em clusters modernos, a sobrecarga de coordenação entre contêineres (latência de rede etc.) também influencia; ferramentas como Kubernetes introduzem pequena latência ao escalar (HPA não é instantâneo, leva segundos para spawn pods). Para demandas ultrarrápidas, podem ser necessárias estratégias de predictive scaling (escalar antes do pico previsto) ou usar arquiteturas serverless complementares.

Outro aspecto interessante é a distribuição geográfica e na nuvem. Contêineres, por serem portáteis, podem ser empacotados e publicados em registries como Docker Hub ou registries privados. Isso permite que ambientes complexos sejam compartilhados facilmente. Por exemplo, um pesquisador pode publicar a imagem do seu experimento de deep learning; outro em outro país pode baixar e rodar exatamente aquele ambiente. Para ciência de dados, isso é transformador: papers com contêineres reproduzíveis.

Não à toa, conferências e journals têm incentivado autores(as) a fornecerem Dockerfiles ou imagens junto com o código dos artigos, aumentando a confiança nos resultados. Projetos como Binder (da Jupyter) usam contêineres para fornecer instâncias de notebooks executáveis de repositórios do Github: o Binder gera uma imagem Docker do ambiente requerido (a partir de requerimentos ou Dockerfile fornecido) e hospeda, permitindo qualquer pessoa executar o notebook de um artigo

sem esforço. Essa iniciativa reforça o valor da containerização para reprodutibilidade científica.

Engenharia de software aplicada à ciência de dados com Docker

Do ponto de vista de melhores práticas de engenharia, o Docker traz para data science conceitos maduros de DevOps e infraestrutura como código. Com ele, o ambiente de execução se torna código (o Dockerfile e arquivos de configuração), que pode ser versionado no Git junto com o código-fonte do projeto. Isso significa que mudanças de ambiente (atualizar uma biblioteca, trocar versão de Python) são rastreáveis e reversíveis como qualquer mudança de código. Essa abordagem disciplinada diminui bastante a famosa frase “na minha máquina tinha pacote X que agora não lembro a versão” – tudo fica documentado declarativamente no Dockerfile.

Além disso, ele habilita CI/CD efetivo para projetos de ciência de dados: podemos ter pipelines automatizados que constroem a imagem Docker e executam testes dentro do contêiner sempre que o novo código é commitado, garantindo que não faltam dependências e que o código roda em um ambiente limpo. Essa filosofia segue o conceito de infraestrutura imutável: em vez de configurar servidores à mão, você recria o ambiente do zero conforme a especificação sempre que necessário, obtendo consistência.

Outro ponto de aproximação com a engenharia de software é a modularidade e a reutilização. Imagens Docker podem ser compostas ou estendidas – por exemplo, você pode construir uma imagem base com ferramentas comuns de ciência de dados (Python + SciPy stack) e depois derivar imagens específicas de projetos adicionando apenas o código e pacotes exclusivos. Isso lembra os princípios de orientação a objetos (herança) ou de componentes reutilizáveis. Times de ciência de dados grandes frequentemente mantêm imagens base padronizadas (com versões aprovadas de certas libs, configurações de proxy etc.), sobre as quais cada novo projeto se baseia. Isso economiza tempo e assegura que todos partem de um ambiente comum testado.

No contexto de controle de versões e experimentação, contêineres permitem congelar exatamente o estado de software de um experimento. É possível, por

exemplo, guardar o hash da imagem Docker usada para treinar um modelo específico. Anos depois, se precisar auditar ou refazer aquele treinamento, você pode rerodar a mesma imagem (Docker garante retrocompatibilidade em formatos), obtendo o mesmo comportamento (supondo que a fonte de dados também esteja disponível). Podemos definir um score de reprodutibilidade – imaginemos hashes representando estado de pacotes, como citado em um guia: a similaridade entre hashes de ambiente esperado vs. ambiente atual pode quantificar reprodutibilidade. De fato, se $H(\text{ambiente_dev}) = H(\text{ambiente_prod})$, podemos dizer que $R \sim 100\%$ (reprodutibilidade perfeita). Essa formalização reforça a ideia de que contêineres garantem H iguais (mesmos pacotes, mesmas versões), coisa que antes era difícil.

Também há sinergia entre Docker e ferramentas de gerenciamento de pacotes ambientes como Conda. Muitas equipes usam Conda inside Docker para gerenciar dependências Python/R com maior controle. Uma prática comum, por exemplo, é manter um `environment.yml` do Conda com todas as libs e usá-lo no Dockerfile (`RUN conda env create -f environment.yml`) para criar o ambiente dentro do contêiner. O Conda lida bem com dependências complexas (incl. binários nativos, MKL, CUDA etc.), agindo como “subgerenciador” dentro da imagem. Um estudo comparativo mostrou que Docker + Conda juntos atingem 99% de reprodutibilidade dos ambientes, muito superior a uso isolado de `virtualenvs/pip`, porque eliminam variâncias de SO e resolvem conflitos de libs nativas.

Ou seja, o Docker cuida do OS e o Conda/pip cuida dos pacotes – união poderosa. No nosso exemplo hands on, usamos pip por simplicidade, mas poderíamos usar Conda se precisássemos de R ou bibliotecas de sistema específicas. Há inclusive projetos como `repo2docker` do Binder que automaticamente geram Dockerfiles a partir de especificações de ambiente (conda, pip etc.).

Em termos de boas práticas de código, ao trabalhar com contêineres os(as) data scientists se beneficiam de pensar em separação de responsabilidades: por exemplo, um container poderia ser apenas responsável por treinamento, outro apenas por servir o modelo. Isso força a estruturar o projeto de modo menos monolítico. Além disso, contêineres incentivam escrever código idempotente e configurável via variáveis de ambiente – evitando trechos dependentes de paths

fixos de máquina ou suposições sobre a presença de determinados softwares. Essa mudança cultural aproxima a ciência de dados de engenharia de software robusta.

Do ponto de vista de sistemas operacionais, o Docker interage intensamente com o kernel para gerenciar processos, mas ao mesmo tempo simplifica para o(a) desenvolvedor(a) muitos detalhes. Memória, CPU, filesystem, tudo isso pode ser gerenciado com comandos Docker em alto nível. Por exemplo, antes se um(a) cientista quisesse limitar seu script a usar 2 CPUs de 16 disponíveis, teria que implementar no código ou usar CGroups manualmente – com Docker, basta `--cpus=2` no run.

Essa abstração de recursos economiza tempo e evita erros. Entretanto, continua importante entender conceitos como limit vs reservation, CPU shares vs quota, para uso eficiente. Uma equação simples frequentemente citada: Utilização = $(\text{CPU_usada}/\text{CPU_total}) + (\text{Mem_usada}/\text{Mem_total})$ – isso para cada contêiner individualmente, indica o quão intensivamente ele consome recursos alocados. Em um cluster orquestrado, manter a soma das utilizações sob controle evita sobrecarga. Ferramentas de monitoramento (Docker stats, cadvisor etc.) fornecem esses números em tempo real, ajudando a ajustar a alocação.

Finalmente, o Docker se encaixa nas práticas modernas de MLOps. Pipelines de CI/CD podem construir imagens ao receber novos modelos e até embutir automaticamente verificações – por exemplo, medir o hash do ambiente do novo modelo e comparar com o anterior para ver se houve deriva (diferença de libs). Muitas empresas já inserem scans de segurança nas imagens (como citamos Hadolint e Trivy) e somente implantam contêineres que passam nos checks. Assim, o deploy de um modelo para produção vira “entrega de um contêiner” – semelhante a entregar um pacote de software tradicional. Essa convergência facilita a vida de equipes multidisciplinares: engenheiros(as) de ML, cientistas de dados e devs falando a mesma língua de containers.

Em suma, contêineres Docker proporcionam aos projetos de ciência de dados: Reprodutibilidade, Portabilidade, Escalabilidade e Manutenibilidade. Isso vem ao custo de adicionar uma camada de complexidade (precisamos aprender Docker e manter Dockerfiles), mas os benefícios superam em muito esse esforço na maioria dos casos, especialmente em ambientes profissionais colaborativos ou de produção. Ao tratar “ambiente como código”, incorporamos o rigor da engenharia ao ciclo de

vida dos projetos de data science, reduzindo surpresas e permitindo foco na ciência de dados em si.

Desde seu lançamento em 2013, o Docker transformou práticas de desenvolvimento – e na ciência de dados não foi diferente. Hoje, contêineres estão onipresentes em fluxos de trabalho de dados em grandes empresas e na academia. Para se ter ideia, cerca de 65% das empresas Fortune 500 já utilizam ambientes containerizados para projetos de IA/ML em 2025, segundo relatórios da IDC.

A habilidade de rodar experimentos consistentemente em diferentes infraestruturas tornou-se tão importante que a proficiência em Docker e DevOps é cada vez mais demandada dos(as) cientistas de dados. A adoção de práticas de MLOps – incluindo containerização – vem crescendo rápido, impulsionada pelo aumento de soluções de IA em produção e pela necessidade de escalar esses sistemas com confiabilidade. Em suma, o Docker virou um padrão de mercado para garantir reprodutibilidade e facilitar o deployment de modelos e pipelines de dados.

Vejamos alguns casos ilustrativos de uso de Docker na ciência de dados:

Reprodutibilidade em pesquisa e educação: o projeto binder do jupyter, já citado, é um caso de sucesso que mostra a importância dos contêineres. Graças ao Binder, qualquer repositório no GitHub com um Dockerfile ou especificações de ambiente pode ser convertido em uma instância interativa online. Isso permitiu, por exemplo, que cursos online e workshops de data science distribuíssem notebooks executáveis sem exigir instalação local – tudo roda em contêineres na nuvem.

Iniciativas de Reproducible Research em universidades também adotaram Docker: na NeurIPS (conferência de ML), houve desafios de reprodutibilidade, incentivando autores(as) a fornecer contêineres com seus experimentos, para que outros(as) pudessem replicar resultados facilmente. Essa tendência melhora a transparência científica e evita “mistérios” sobre por que o código do paper não roda na máquina de quem tenta reproduzir.

Plataformas internas de empresas: empresas de tecnologia implantaram plataformas de ciência de dados baseadas em contêineres. A Netflix, por exemplo, conhecida por seu ótimo uso de contêineres em microsserviços, também aplica Docker no ambiente de dados – informações públicas sugerem que a Netflix usa Jupyter notebooks containerizados para que engenheiros(as) e pesquisadores(as)

executam análises em clusters de forma consistente e utiliza Kubernetes para escalar jobs de treinamento e tunagem de algoritmos de recomendação. Outro case é a Uber, cujo sistema Michelangelo (plataforma de ML end-to-end) faz heavy use de contêineres para isolar jobs de treinamento e servir modelos em escala global (cada modelo é embalado em um container para deploy contínuo em vários datacenters). Essas empresas relatam melhorias drásticas em tempo de deploy e redução de erros de configuração ao adotar Docker em seus pipelines de ML.

Comunidade open source e startups: ferramentas populares de data science foram disponibilizadas como imagens Docker para facilitar a vida de practitioners. Por exemplo, existem imagens oficiais como jupyter/scipy-notebook contendo JupyterLab + principais libs de ciência de dados em Python, prontas para uso. Isso significa que em poucos segundos qualquer pessoa pode subir um ambiente completo via docker run sem instalar nada – excelente para workshops ou protótipos. Startups de visão computacional e NLP com times enxutos frequentemente adotam Docker desde o começo, permitindo que o mesmo artefato (imagem) vá do laptop para a nuvem para o cliente, agilizando entregas.

Por outro lado, startups que ignoraram a reprodutibilidade às vezes aprenderam da pior forma: há casos de equipes que implantaram modelos manualmente em produção e, ao precisar escalar, enfrentaram semanas de retrabalho porque era difícil recriar exatamente o ambiente em novos servidores – algo que poderia ter sido trivial com containers. Esse tipo de “case de fracasso” (não divulgado publicamente, mas comum nos bastidores) reforça porque até pequenos negócios estão investindo em containerizar suas soluções de dados o quanto antes.

Séries e conteúdos on-line: a difusão do Docker no meio de dados também se reflete na quantidade de materiais educativos disponíveis. Por exemplo, a plataforma DataCamp publicou tutoriais em português como “Como aprender Docker do zero: um guia para profissionais de dados”, orientando iniciantes em todos os passos (instalação, comandos básicos, melhores práticas) especificamente voltados ao uso em ciência de dados. No YouTube, há cursos e playlists dedicados – desde workshops práticos como “Docker do Zero ao Deploy” (por Alan Lanceloth) até aulas em universidades.

Um exemplo acadêmico é a série Engenharia e Ciência de Dados Cidadã 2025 da Unicamp, que inclui uma aula demonstrando Docker com PostgreSQL

aplicado a dados reais. Esses conteúdos mostram, na prática, casos de uso combinando contêineres com bancos de dados, notebooks e pipelines, ajudando profissionais e estudantes a se capacitarem na ferramenta.

Também livros como “Docker in Action” (Nickoloff & Kuenzli, 2019) tornaram-se referências – cobrindo desde fundamentos do Docker até como aplicar em cenários do mundo real. Esse livro enfatiza que o Docker encapsula aplicações e dependências em ambientes leves e isolados, facilitando instalação, distribuição e escalabilidade de aplicativos. Para cientistas de dados, obras assim fornecem base sólida para entender não apenas o “como”, mas o “porquê” de se usar contêineres.

Falando em tendências, o ecossistema de containerização continua evoluindo e impactando a área de dados. Uma tendência é a busca por contêineres ainda mais leves e seguros: projetos como Distroless (imagens Docker sem shell ou gerenciador de pacotes, reduzindo a superfície de ataque) estão ganhando adoção – por exemplo, contêineres apenas com runtime Python e libs necessários, nada mais.

Isso melhora a inicialização e a segurança, importante para deploy serverless de modelos em que cada milissegundo conta. Outra tendência é a conteneirização rootless – ferramentas como Podman e o próprio Docker rootless mode permitem rodar contêineres sem privilégios de superusuário, aumentando a segurança em ambientes multi-tenant (imagine vários usuários estudantes rodando contêineres no mesmo servidor – o rootless impede escapes maliciosos).

No front de orquestração, vê-se uma consolidação do Kubernetes como “sistema operacional da nuvem” – muitos serviços gerenciados (AWS ECS, Google GKE, Azure ML) oferecem camadas por cima de K8s para facilitar deploy de modelos contêinerizados sem que o(a) cientista de dados precise saber detalhes de K8s, mas vale a pena ter noção de como funciona para tuning fino. Em MLOps, frameworks novos estão integrando o Docker nativamente: por exemplo, o MLflow pode registrar um modelo e exportá-lo como imagem Docker automaticamente para deployment; o Kubeflow cria pipelines em que cada etapa é um contêiner. Há também avanços em ferramentas de snapshotting de contêiner (CRI-O, Containerd) para tornar o overhead ainda menor.

Uma tendência interessante é a fusão de gerenciamento de pacotes e containers – a comunidade Python, por exemplo, discute formatos como PEX ou

conda-pack, mas até agora nada supera o Docker em isolar também o sistema. Uma solução híbrida ganhando espaço é o uso de Mamba (um resolver de dependências ultrarrápido) junto com Docker, para agilizar builds de imagens com conda.

Ou seja, atacar o problema de ambientes reproduzíveis por todos os lados: resolução eficiente das deps + containerização. No domínio de HPC e AI research, contêineres especializados como Singularity/Apptainer são tendência para executar workloads em supercomputadores e GPUs de forma segura (pois Singularity não requer privilégios e é compatível com imagens Docker). Isso indica que mesmo em ambientes mais tradicionais (HPC científico) a containerização se firmou (CHOUDHARY, 2025).

No geral, podemos afirmar que saber usar Docker tornou-se parte do arsenal do(a) cientista de dados moderno(a). Os casos de sucesso mostram ganhos de produtividade, confiabilidade e colaboração. Os casos de insucesso (quando ignoram reprodutibilidade) servem de alerta: com datasets e modelos crescendo em complexidade, depender de configurações manuais não escala. Como todo conceito, há limites – contêineres não corrigem código ruim nem garantem por si só performance ótima – mas formam um alicerce sólido sobre o qual construímos soluções de data science robustas.

A recomendação de especialistas é clara: para usar Docker de forma eficaz, vale planejar a arquitetura de seus projetos pensando em separação de componentes (dados, treinamento, servir modelo etc.) e adotar contêineres sempre que precisar mover código entre ambientes distintos ou replicar execuções futuramente. Praticamente, isso significa quase sempre projetos colaborativos ou produtos de dados. Em contrapartida, se você está explorando rapidamente em um ambiente local controlado, contêineres podem inicialmente parecer overhead – mas ainda assim, colocar seu experimento em um container quando ele estabiliza traz enormes benefícios, permitindo que outros rodem e que você mesmo escale ou reutilize no futuro.

Para concluir, o Docker inaugurou uma era em que “ambiente não é mais obstáculo”. Na pós-graduação e no mercado, cientistas de dados capazes de dominar essa ferramenta conseguem transformar notebooks experimentais em aplicações reprodutíveis de maneira muito mais tranquila. De laboratórios acadêmicos que publicam contêineres junto com artigos, até times de data science

entregando modelos continuamente em produção global, a containerização está na base dessas conquistas.

A tendência é que esse movimento se intensifique, com contêineres cada vez mais integrados a fluxos de ML (inclusive com foco em DevSecOps – segurança integrada). Em suma, saber quando e como usar Docker de forma eficiente tornou-se parte essencial do toolkit do(a) profissional de dados e evita inúmeros problemas clássicos (como incompatibilidades de ambiente e dificuldades de escalar código). Nas raras situações em que o Docker pode não ser a melhor escolha – por exemplo, para pequenas tarefas locais extremamente simples ou scripts one-off – ainda assim entender os princípios que ele traz (isolamento, definição declarativa de ambiente) ajudará a conduzir o trabalho de forma organizada.

O panorama atual indica que o Docker e contêineres vieram para ficar na Ciência de Dados, impulsionando colaboração e inovação sem as barreiras de “em qual máquina isso funciona?”. Portanto, embarque nessa tecnologia e leve seus projetos ao próximo nível – como se diz no jargão: “containerize all the things!”. Afinal, ambiente reproduzível não é luxo, é requisito básico para ciência (de dados) de qualidade.

MERCADO, CASES E TENDÊNCIAS

O uso de contêineres Docker na ciência de dados tornou-se uma prática consolidada em empresas, universidades e comunidades open source. Segundo a IDC, cerca de 65% das empresas da Fortune 500 já utilizam ambientes containerizados para projetos de IA e ML (Machine Learning), evidenciando a maturidade da tecnologia no mercado corporativo. Plataformas como Binder (Jupyter) e MLflow integram Docker para garantir reprodutibilidade e facilitar o deploy de modelos.

Cases ilustrativos:

- **Netflix** utiliza contêineres para orquestrar notebooks Jupyter e pipelines de recomendação em escala global.
- **Uber** emprega Docker no sistema Michelangelo para isolar jobs de treinamento e servir modelos em múltiplos datacenters.
- **Unicamp** publicou a série “Engenharia e Ciência de Dados Cidadã 2025”, com aulas práticas sobre Docker e PostgreSQL aplicados a dados reais.

Publicações e vídeos abertos:

- [“Docker para Ciência de Dados – Guia Passo a Passo”](#).
- [“Descomplicando o Docker” – Marcelo Rodrigues, DIO](#).
- [“Docker + IA: Acabei com o ‘funciona na minha máquina’” – Sergio Monteiro](#).
- [“Docker e Reprodutibilidade” – Workshop NeurIPS](#).

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos como contêineres Docker resolvem o problema da reprodutibilidade em ciência de dados, eliminando o clássico “funciona na minha máquina”. Aprendemos a construir imagens Docker com Python e bibliotecas específicas, executar contêineres com volumes e redes e aplicar boas práticas para segurança e eficiência.

Aprofundamos conceitos de virtualização, controle de recursos, escalabilidade e integração com sistemas operacionais. Por fim, analisamos casos reais e tendências que mostram como o Docker está transformando o mercado de ciência de dados.

REFERÊNCIAS

CHOUDHARY, A. **DevOps for Data Science**: Reproducible Environments with Docker and Conda. 2025. Disponível em: <https://johal.in/devops-for-data-science-reproducible-environments-with-docker-and-conda-2025/>. Acesso em: 15 dez. 2025.

MONTEIRO, S. A. **Docker + IA**: Acabei com o “funciona na minha máquina”. 2025. Disponível em: <https://pt.linkedin.com/pulse/docker-ia-acabei-com-o-funciona-na-minha-m%C3%A1quina-assun%C3%A7%C3%A3o-monteiro-sipwf>. Acesso em: 15 dez. 2025.

NICKOLOFF, J.; KUENZLI, S. **Docker in Action** – Second Edition. Shelter Island: Manning Publications, 2019.

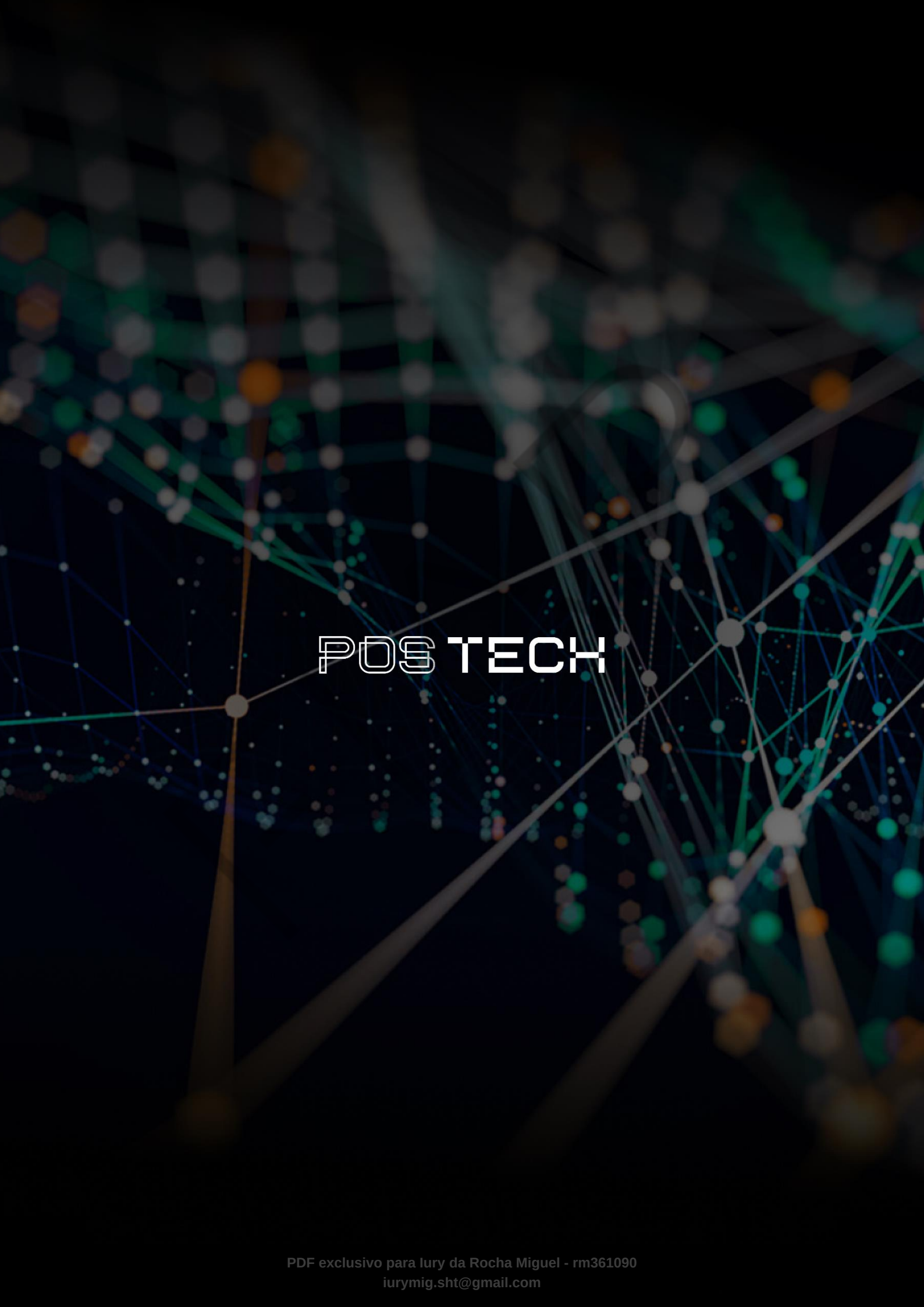
RODRIGUES, M. **Descomplicando o Docker**: Como Contêineres Estão Revolucionando o Deploy. 2025. Disponível em: <https://www.dio.me/articles/descomplicando-o-docker-como-containers-estao-revolucionando-o-deploy-1c7b7cf19599>. Acesso em: 15 dez. 2025.

VICTORINO, V. **Docker**: “funciona na minha máquina”. 2022. Disponível em: <https://viniciusflv.github.io/pt-BR/posts/devops/docker/>. Acesso em: 15 dez. 2025.

PALAVRAS-CHAVE

Reprodutibilidade computacional. Containerização. Engenharia de software para ciência de dados.

EMENDAS



POSTECH